ALU–FPU Parallelism on ARM Cortex-A76: A Performance Study

CS2323 Computer architecture

Final project report

-CS24BTECH11060 T. AMRUTHA
-CS24BTECH11038 M. REETHIKA

Abstract

Modern ARM architectures such as the Cortex-A76 contain highly capable parallel execution units including integer ALUs, floating-point pipelines, and NEON SIMD pipelines. While integer programs naturally utilize ALUs, floating-point and NEON units may remain underutilized. This project studies whether converting selected integer computations to floating-point arithmetic or splitting computation between ALU and FPU can yield performance benefits. We evaluate several non-auto-vectorizable functions: polynomial evaluation, nCr computation, cumulative summation, factorial, binary search, and prime counting. For each, we test strategies such as dependency breaking, loop unrolling, mixed integer-float execution, and FPU heavy restructuring. Our results show that ALU FPU parallelism provides benefits only when *dependencies are breakable* and auto vectorization does not already saturate FP/NEON pipelines. Certain functions (e.g., factorial, polynomial evaluation) achieved up to 58% improvement, while others, limited by strict dependencies or auto vectorization, gained little or regressed.The study identifies the specific scenarios where ALU FPU parallelism becomes effective.

➔ Introduction

Integer-based C programs typically rely heavily on the ALU pipeline, leaving the floating-point unit (FPU) underutilized. Modern processors like the ARM Cortex-A76 are superscalar, allowing integer, floating-point, load/store, and branch instructions to run in parallel when they are independent. This opens the question of whether restructuring purely integer algorithms into mixed ALU/FPU versions can reduce execution time by exploiting otherwise idle hardware.

Another important factor is compiler auto-vectorization. NEON SIMD pipelines may perform parts of the computation using FP/NEON hardware even for integer code, potentially limiting the benefit of manually introducing floating-point parallelism.

This project therefore focuses on understanding the conditions under which ALU–FPU concurrency helps, the cases where dependencies restrict parallelism, and how auto-vectorization influences the actual speedups that can be achieved.

➔ Hardware Background (ARM Cortex-A76)

The ARM Cortex-A76 contains:

- Scalar ALUs for integer arithmetic
- Scalar floating-point pipelines
- NEON SIMD pipelines inside the FP/NEON execution block
- Load/store pipelines
- Branch execution units

Important properties:

- `-O3` aggressively auto-vectorizes whenever possible.
- Auto-vectorization uses NEON registers (v0, v1, …), hence FP/NEON pipelines may already be active even in pure integer code.
- The FPU supports FMA instructions, reducing multiply + add into one pipelined step.

Reference : Technical reference manual for ARM Cortex A-76.

The Cortex-A76 core includes the following features:

Core features

- 40-bit *Physical Address* (PA).
- A *Memory Management Unit* (MMU).
- Optional Cryptographic Extension.
- Armv8.4 Dot Product instruction support.
- Superscalar, variable-length, out-of-order pipeline.
- Support for Arm TrustZone® technology.
- Support for *Page-Based Hardware Attributes* (PBHA).
- *Reliability, Availability, and Serviceability* (RAS) Extension.
- Full implementation of the Armv8.2-A A64, A32, and T32 instruction sets.
- *Generic Interrupt Controller* (GICv4) CPU interface to connect to an external distributor.
- Generic Timers interface supporting 64-bit count input from an external system counter.
- An integrated execution unit that implements the Advanced SIMD and floating-point architecture support.
- AArch32 Execution state at Exception level EL0 only. AArch64 Execution state at all Exception levels (EL0 to EL3).

➔ Methodology

❖ Checking Auto-Vectorization

Auto-vectorization was detected by:

- Inspecting generated .s assembly files
- Identifying NEON registers (e.g., v0.4s, v1.2d)

If NEON registers appear, the FP/NEON unit is executing work, even when the C source uses only integers.

❖ Performance Measurement

- Used `clock_gettime(CLOCK_MONOTONIC_RAW)` for stable timing.
- Large inputs were generated using C programs to maintain consistency.
- For noisy measurements, a median of 5 runs was used.

➔ Selected Functions

The following were analyzed:

- Polynomial evaluation (Horner's method)
- NCR computation
- Cumulative array summation
- Binary search
- Factorial
- Prime number count
- Matrix multiplication

➔ Experiments and Analysis:

1) Polynomial evaluation using Horner's method:

a) Base Integer Version

The integer version computes the polynomial using Horner's rule:
This loop has strict loop-carried dependency. The next iteration cannot begin until the previous multiplication and addition finish. Therefore:

- No auto-vectorization occurs
- FPU remains completely idle
- Only ALU performs the computation

```
    clock_t start = clock();
        //given d is the degree of polynomial
    long long result = coeff[0];
    for (long long i = 1; i <= d; i++) {
        result = result * x + coeff[i];
    }
    clock_t end = clock();
```

Why Conversion Was Needed

To utilize the idle FPU, the polynomial must be split such that independent segments can be computed in parallel. Horner's method does not allow this, so we divide the polynomial into two or more segments.

b) k-Splitting (Divide Polynomial into Parts)

We divide polynomial of degree *d* into two independent halves:

- FPU computes higher-degree part (`r1`)
- ALU computes lower-degree part (`r2`)

Reasoning:

- These two halves are independent
- ALU and FPU can run in parallel
- Only one final multiplication by `x^(mid+1)` is needed

```
        long long mid = d / 64;
clock_t start1 = clock();
        double r1=0.0;
        for(long long i=d;i>=mid+1;i--){
                r1=fma(r1,x,coeff[i]);
        }
        double xp=pow(x,mid+1);
clock_t end1 = clock();
clock_t start2=clock();
        long long r2=0;
        for(long long i=mid;i>=0;i--){
                r2=r2*x+coeff[i];
        }
        double r2f=(double)r2;
        double result=fma(r1,xp,r2);
clock_t end2=clock();
```

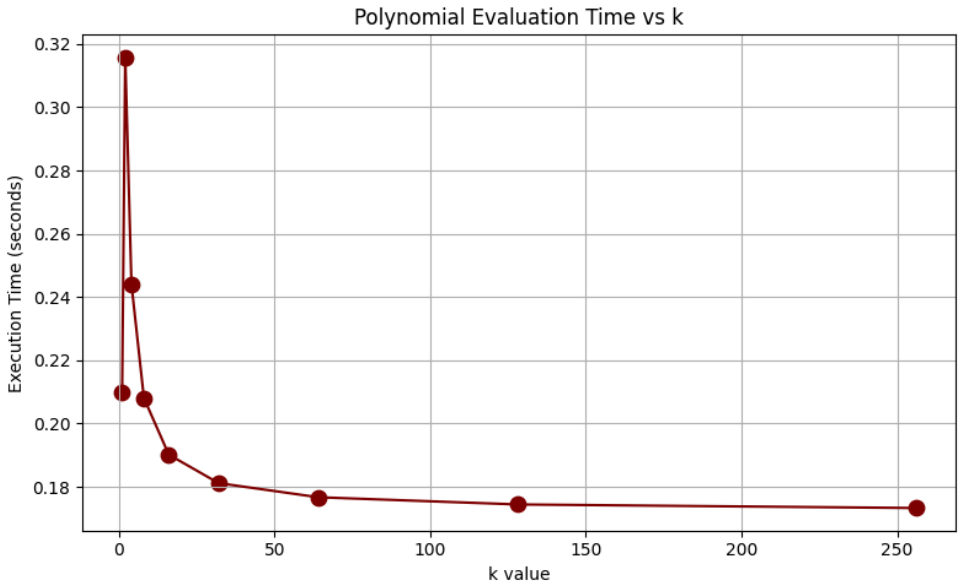
We tested multiple values: k = 2, 4, 8, 16, 32, 64, 128, 256.

Reason:

- Loop counters and address calculations still require ALU
- Giving more arithmetic work to FPU reduces ALU bottleneck

Observations

- For small k, time increased (FPU overhead > saved ALU time)
- For k ≥ 64, performance saturated (FPU fully utilized)



Polynomial split improves performance by **19.115%**

2) nCr (n choose r) calculation

a) Integer version

- Computes nCr by iteratively updating `result = result * (n - r + i) / i` inside a loop for `i = 1..r`.
- No auto vectorization because of dependency in the loop.

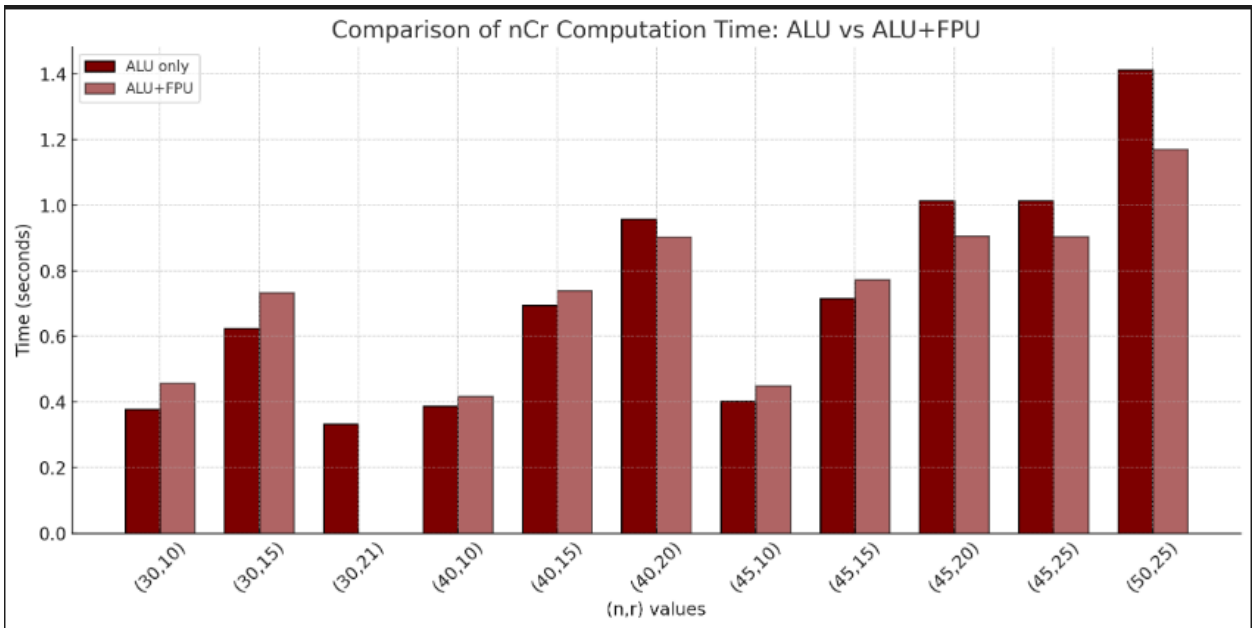
```
double start = now();
for (long long int k = 0; k < repeats; k++){
        result = 1;
        dummy += 1;
        for (int i = 1; i <= r; i++) {
            result = result * (n - r + i) / i;
        }
}
double end=now();
```

b) ALU-FPU version

- This version split the multiplicative product over `r` into two independent halves.

```
double start = now();
for (long long rep = 0; rep < repeats; rep++) {
    long long alu_tmp = 1;
    double fpu_tmp = 1.0;

    for (int i = 1; i <= fpuCount; i++) {
        if (i <= mid) {
            long long num = (n - r + i);
            alu_tmp = (alu_tmp * num) / (i);
        }

        fpu_tmp = (fpu_tmp * fpuNum[i-1]) / fpuDen[i-1];
    }

    result1 = alu_tmp;
    result2 = fpu_tmp;
    dummy++;
}

double end = now();
```

Observations:



- When r is < 20 then the number of iterations are small, there are not many independent instructions to overlap alu and fpu pipelines, so the time remains same or even increases because of fpu latency.
- when r is > 20 then the number of independent instructions are sufficient to overlap tha alu and fpu pipelines so the time decreases

3) Cumulative summation of elements of a given array

a) Base Integer Version

A completely dependent cumulative-sum loop:

- Next iteration waits for previous
- No auto-vectorization

```
 double start_i = now();
for(int r=0; r < repeats; r++){
    for (int i = 0; i < m; i++)A[i] = B[i];
//here B[] is just used to prevent optimisation to see the time
    for (int i = 1; i < m; i++)A[i] += A[i-1];
    dummy++;
}
    double end_i = now();
```

To break dependency, an accumulator form was tried.

b) Accumulator version

```
 double start_i = now();
for(int r=0; r < repeats; r++){
    sum=0;
    for (int i = 1; i < m; i++)sum += A[i];
    dummy++;
}
    double end_i = now();
```

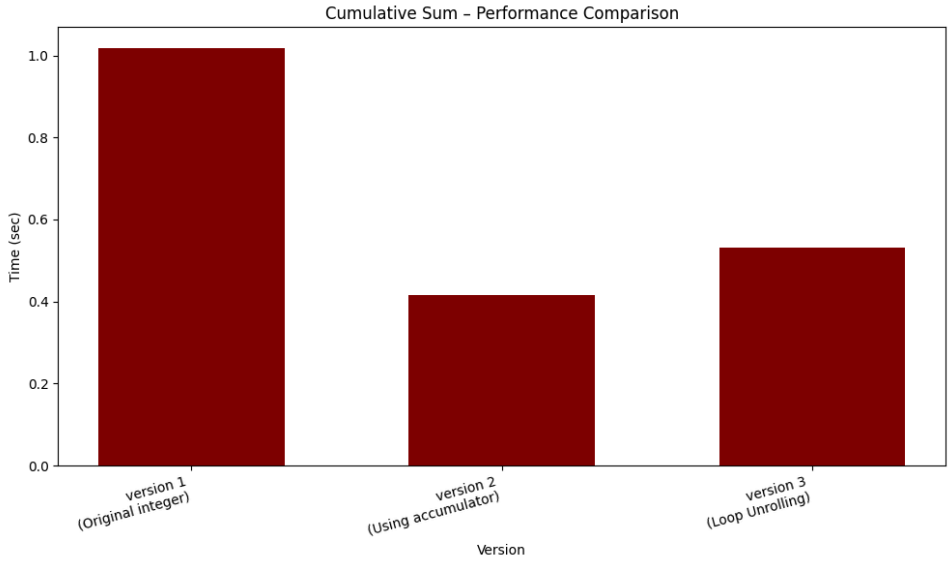
Surprisingly, the compiler produced **2-lane SIMD** because integer addition is associative.

c) Loop Unrolling

```
double start_i = now();
for(int r=0; r < repeats; r++){
        sum=0;
        fsum=0.0;
        for (int i = 1; i < m; i+=2){
                sum += A[i];
                fsum+= A[i-1];
        }
    dummy++;
}
double end_i = now();
printf("sum=%lld\n",sum+(long long)fsum);
```

Loop unrolling changed the loop structure and disabled auto-vectorization because the compiler cannot guarantee associativity under floating-point rounding.

Observations



Conclusion

- Auto-vectorized version => **59.158%** faster
- Unrolled version => **47.874%** faster, but slower than SIMD Integer

4) Binary search

a) Base integer version

```
double start = now();
for(long long int i=0 ;i<repeats;i++){
 l = 0;
     r = n - 1;
 ans = -1;
 dummy += 1;
     while (l <= r) {
        long long int mid = l + (r - l) / 2;

        if (arr[mid] == target) {
                ans = mid;
                break;
        } else if (arr[mid] < target) {
                l = mid + 1;
        } else {
                r = mid - 1;
        }
    }
 }
double end = now();
```

Problems With Binary Search (Why ALU–FPU Parallelism Fails)

Binary search has a strict dependency chain: each iteration needs the updated l, r and mid from the previous iteration before it can continue. Because of this, neither the ALU nor the FPU can start the next computation early, they must wait for the previous comparison to complete.
Even if we try to move the mid-calculation to the FPU, the int↔float conversion adds extra instructions and makes the loop slower. The algorithm is also branch-heavy, meaning mispredictions frequently flush the pipeline and reduce instruction-level parallelism. For these reasons, a **single binary search cannot benefit from ALU FPU parallelism**.

How Multiple Binary Searches Can Work in Parallel

While one binary search is strictly sequential, **multiple binary searches running at the same time can be parallelized**. For example, if we search for several keys in the same sorted array, we can assign some searches to the ALU (pure integer mid computation) and others to the FPU (float-based mid computation).
Here, each search has its own independent (l,r,mid) chain, so the CPU can execute ALU instructions for one search while FPU instructions for another search run in parallel. This breaks the dependency at the problem level, not at the instruction level, allowing parallel ALU+FPU utilization. However, this works only when **multiple independent queries** exist, not for a single search.

5) Factorial calculation

a) Base integer version

```
double start=now();
        for(int r=0;r<repeats;r++){
                p=1;
                for(int i=1;i<=n;i++){
                        p*=i;
                }
                dummy++;
        }
double end=now();
```

b) 2-Way Unrolling (ALU + FPU)

- ALU multiplies even numbers
- FPU multiplies odd numbers in parallel.

```
double start=now();
        for(int r=0;r<repeats;r++){
                p=1;
                pf=1.0;
                for(int i=2;i<=n;i+=2){
                        p*=i;
                        pf*=(i-1);
                }
                dummy++;
        }
        long long ans=p*(long long)pf;
        if(n%2==1) ans*=n;
double end=now();
```

c) 3-Way Unrolling

- One ALU stream
- Two FPU streams

```
double start=now();
        for(int r=0;r<repeats;r++){
                p=1;
                pf1=1.0;
                pf2=1.0;
                for(int i=3;i<=n;i+=3){
                        p*=i;
                        pf1*=(i-1);
                        pf2*=(i-2);
                }
                dummy++;
        }
        long long ans=p*(long long)pf1*(long long)pf2;
        if(n%3==1) ans*=n;
        else if(n%3==2) ans = ans*n*(n-1);
double end=now();
```

d) 4-Way Unrolling

```
double start=now();
        for(int r=0;r<repeats;r++){
                p1=1;
                p2=1;
                pf1=1.0;
                pf2=1.0;
                for(int i=4;i<=n;i+=4){
                        p1*=i;
                        p1*=(i-1);
                        pf1*=(i-2);
                        pf2*=(i-3);
                }
                dummy++;
        }
        long long ans=p1*p2*(long long)pf1*(long long)pf2;
        if(n%4==1) ans*=n;
        else if(n%4==2) ans = ans*n*(n-1);
        else if(n%4==3) ans=ans*n*(n-1)*(n-2);
double end=now();
```

4 way loop unrolling gave less improvement when compared to 3 way loop unrolling due to increased register pressure in ALU.

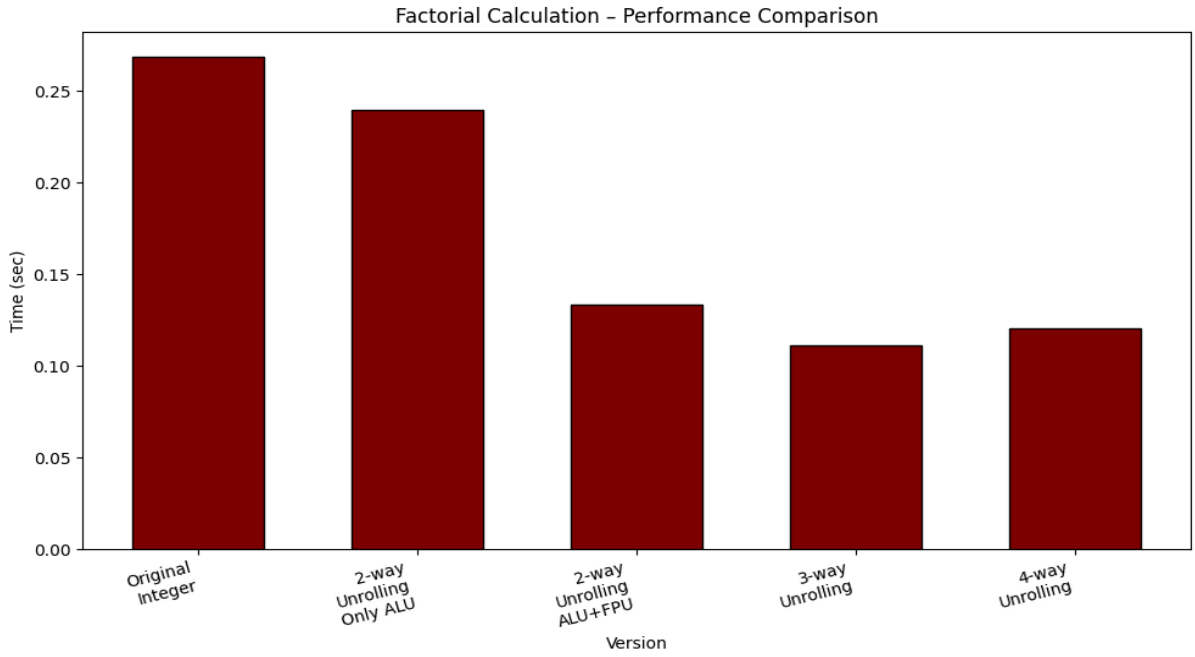
e) 2-way unrolling (Only ALU)

To verify whether the speedup observed in the factorial experiment was coming from loop unrolling itself or from ALU–FPU parallelism, we separately tested a 2-way unrolled version using pure integer code.

```
double start=now();
        for(int r=0;r<repeats;r++){
                p=1;
                p2=1;
                for(int i=2;i<=n;i+=2){
                        p*=i;
                        p2*=(i-1);
                }
                dummy++;
        }
        long long ans=p*p2;
        if(n%2==1) ans*=n;
double end=now();
```

Here in this case there is improvement in performance in terms of time, but not more improvement when compared to 2 way loop unrolling with ALU+FPU.

Observations



Conclusion

2 way unrolling using ALU and FPU => **50.427%** faster
2 way unrolling only using ALU => **10.704%** faster
3 way unrolling => **58.564%** faster
4 way unrolling => **55.251%** faster

6) Prime number count

a) Base integer version

- This version counts the number of prime numbers in a given range using only alu.

- No auto vectorization.

```
double begin = now();
for (long long num = start; num <= end; num++) {

    int isPrime = 1;

    if (num <= 1) {
        isPrime = 0;
    }
    else if (num <= 3) {
        isPrime = 1;
    }
    else if (num % 2 == 0 || num % 3 == 0) {
        isPrime = 0;
    }
    else {
        for (long long i = 5; i * i <= num; i += 6) {
            if (num % i == 0 || num % (i + 2) == 0) {
                isPrime = 0;
                break;
            }
        }
    }

    if (isPrime)
        count++;
}
double completed = now();
```

b) ALU-FPU Version 1

- This version performs prime counting in parallel by splitting the range into two halves

```
double begin = now();

while(num_int <= mid || num_double <= (double)end) {

    if(num_int <= mid) {
        int isPrime = 1;

        if(num_int <= 1) isPrime = 0;
        else if(num_int <= 3) isPrime = 1;
        else if(num_int % 2 == 0 || num_int % 3 == 0) isPrime = 0;
        else {
            for(long long i = 5; i*i <= num_int; i += 6) {
                if(num_int % i == 0 || num_int % (i+2) == 0) {
                    isPrime = 0;
                    break;
                }
            }
        }

        if(isPrime) count++;
        num_int++;
    }

    if(num_double <= (double)end) {
        int isPrimeF = 1;
        double nd = num_double;

        if(nd <= 1.0) isPrimeF = 0;
        else if(fp_mod(nd, 2.0) == 0.0 || fp_mod(nd, 3.0) == 0.0) isPrimeF = 0;
        else {
            for(double i = 5.0; i*i <= nd; i += 6.0) {
                if(fp_mod(nd, i) == 0.0 || fp_mod(nd, i+2.0) == 0.0) {
                    isPrimeF = 0;
                    break;
                }
            }
        }

        if(isPrimeF) count++;
        num_double += 1.0;
    }
}

double completed = now();
```

c) ALU-FPU Version 2(loop unrolling)

- This version computes the number of prime numbers in parallel using alu and fpu by loop unrolling.

```
double begin = now();

while(num <= end) {
    if(num <= end) {
        int isPrime = 1;

        if(num <= 1) isPrime = 0;
        else if(num <= 3) isPrime = 1;
        else if(num % 2 == 0 || num % 3 == 0) isPrime = 0;
        else {
            for(long long i = 5; i*i <= num; i += 6) {
                if(num % i == 0 || num % (i+2) == 0) {
                    isPrime = 0;
                    break;
                }
            }
        }

        if(isPrime) count++;
    }

    if(num + 1 <= end) {
        double nd = (double)(num + 1);
        int isPrimeF = 1;

        if(nd <= 1.0) isPrimeF = 0;
        else if(fp_mod(nd, 2.0) == 0.0 || fp_mod(nd, 3.0) == 0.0) isPrimeF = 0;
        else {
            for(double i = 5.0; i*i <= nd; i += 6.0) {
                if(fp_mod(nd, i) == 0.0 || fp_mod(nd, i+2.0) == 0.0) {
                    isPrimeF = 0;
                    break;
                }
            }
        }

        if(isPrimeF) count++;
    }

    num += 2;
}

double completed = now();
```

Conclusion:

- There is no improvement in time.
- Even though alu and fpu work in parallel, the latency of the fp_mod function is very high so there is no improvement in performance.
- The fp_mod function involves floating point multiplication, division, subtraction and type casting, so the latency of the function is very high.

7) ***Matrix multiplication***

a) *Base integer version*

```
clock_t start = clock();
        for(int i=0;i<m;i++){
                for(int j=0;j<p;j++){
                        for(int k=0;k<n;k++){
                                r[i][j]+=a[i][k]*b[k][j];
                        }
                }
        }
 clock_t end = clock();
```

Base integer version Triply nested loop. The compiler with -03 aggressively auto-vectorizes this loop using NEON registers. Even though the code is written using integers, the generated assembly contains SIMD (`v0.4s,v1.4s,.....`) instructions. This means the FP/NEON execution block is already active.

Because of this, ALU FPU parallelism may not provide extra benefit FPU/NEON pipelines are already busy.

```
.L18:
    ldr s5, [x0, x26]
    ldr s0, [x0]
    ldr s4, [x0, x7]
    ldr s3, [x0, x5]
    add x0, x0, x6
    ins v0.s[1], v5.s[0]
    ldr q2, [x1], 16
    ins v0.s[2], v4.s[0]
    ins v0.s[3], v3.s[0]
    mla v1.4s, v0.4s, v2.4s
    cmp x4, x1
    bne .L18
    addv    s0, v1.4s
    fmov    w0, s0
    add w2, w2, w0
    mov w0, w14
    cmp w10, w14
    beq .L19
```

```
.L17:
```

b) Cache blocking (tiling)

To reduce cache misses for large matrices (e.g., 800×900), we applied cache blocking.

Blocking improves performance because:

- Each block fits into L1/L2 cache
- Elements of A and B are reused

```
        clock_t start=clock();

        for(int ii=0;ii<m;ii+=block){

                for(int jj=0;jj<p;jj+=block){

                        for(int kk=0;kk<n;kk+=block){

                                for(int i=ii;i<ii+block&&i<m;i++){

                                        for(int
j=jj;j<jj+block&&j<p;j++){

                                                float sum=0.0f;

                                                for(int
k=kk;k<kk+block&&k<n;k++){


sum+=(float)a[i][k]*(float)b[k][j];

                                                }

                                                r[i][j]+=(int)sum;

                                        }

                                }

                        }

                }
```

```
        }

        clock_t end=clock();
```

Combination of various versions i.e manual SIMD vectorization, cache blocking will help more effectively.

Cache blocking (int): **1.11× faster**

Cache blocking (float): **1.32× faster**

NEON SIMD + blocking + FMA: **4.48× faster**

These versions are avoided since this is not the scope of the project.

➔ General guidelines for ALU FPU Parallelism

- Prefer splitting independent computation.
- Reduce ALU load by shifting arithmetic to FPU where possible.
- Break dependencies using loop unrolling or splitting.
- Avoid repeated int ⇔ float conversions.
- Check for auto-vectorization,if active, FP/NEON may already be saturated.

➔ Challenges Faced

- **Time instability:** Execution time changed across runs because the OS scheduled background tasks, the CPU temperature changed, or the frequency changes. This made accurate measurement harder, especially for small inputs.

- **Cache behaviour:** The first few runs always suffered from cache effects. Large arrays and matrices often caused compulsory cache misses, which added noise and masked small performance improvements.

- **Conversion overhead:** Whenever we converted integers to floats and back, the compiler inserted extra instructions. In many cases, these conversions took more time than the work we tried to offload to the FPU.

- **Precision limitations:** Floating-point arithmetic caused small rounding errors that accumulated in some functions. This limited how aggressively we could shift work to the FPU.

- **Dependency chains:** Many functions had strong loop-carried dependencies, meaning the next iteration could not start until the current one finished. This blocked parallel execution even if both ALU and FPU were free.
- We tried to **measure power** but failed.

➔ Limitations

- Methods fail for tightly sequential loops.
- Float precision may introduce small rounding errors.
- Auto-vectorization sometimes outperforms manual ALU–FPU techniques.
- Heavy unrolling increases register pressure.
- Results in large code size.

➔ Key Learnings

- ALU FPU parallelism is beneficial only when dependencies are breakable.
- Auto-vectorization significantly alters achievable performance.
- Loop unrolling offers major benefits only when used carefully.
- Some algorithms inherently cannot be accelerated by ALU FPU splitting.

➔ Conclusion

This project demonstrates that accelerating integer C code by adding floating-point operations or splitting computation between ALU and FPU is effective only under specific structural conditions. When loops are independent or partially independent, and when auto-vectorization does not already occupy FP/NEON pipelines, significant improvements can be achieved (up to ~58%). For tightly dependent loops and already vectorized code, ALU–FPU parallelism offers little or no benefit. Understanding compiler behavior, dependency patterns, and pipeline characteristics is crucial for effective performance engineering on modern ARM architectures.

Link for codes and README:

https://drive.google.com/file/d/1g3uBm5ALk5rlXhOBDUGlGf861p1O8Kx-/view?usp=sharing